Kubernetes-Based Canary Deployment with K3s and Istio

Introduction

Canary deployment is a modern strategy to release new application versions incrementally while minimizing risk.
It allows traffic to be gradually shifted from the stable version to a new release, ensuring smooth rollouts and fast rollback if issues arise.
This project demonstrates a practical implementation of canary deployment using lightweight Kubernetes (K3s) and Istio service mesh.

Abstract

The objective of this project was to simulate a canary deployment scenario for a Python-based application with two versions.
By leveraging Istio's traffic management capabilities, we split incoming requests between stable and canary versions, monitored performance, and adjusted traffic based on metrics.
This approach provides a safe, automated mechanism to release updates while maintaining application reliability.

Tools Used

- K3s - Lightweight Kubernetes distribution for easy local cluster setup
- Istio - Open-source service mesh for traffic routing and observability
- Docker - Containerization of application versions
- Helm - Simplified deployment and configuration management
- Python App - Sample application with multiple versions
- Prometheus & Grafana (optional) - For monitoring traffic and performance

Steps Involved in Building the Project

1. Environment Setup: Installed K3s and configured a local Kubernetes cluster.
2. App Containerization: Built Docker images for Python app versions v1 (stable) and v2 (canary).
3. K3s Deployment: Created YAML manifests to deploy both versions in Kubernetes, including deployments and services.
4. Istio Configuration:
   - Installed Istio and enabled sidecar injection.
   - Created Gateway and VirtualService YAML files to split traffic (80% to stable, 20% to canary).
5. Traffic Monitoring: Observed request distribution using logs or a metrics dashboard.
6. Canary Management: Based on performance metrics, traffic was gradually shifted to the new version or rolled back in case of issues.

Conclusion

This project demonstrates an effective canary deployment strategy in a lightweight Kubernetes environment using Istio.
The traffic splitting approach ensures minimal disruption while testing new versions.
The process can be scaled to larger environments and integrated with CI/CD pipelines for automated deployment, ensuring safer and faster releases in production environments.